

Analytical Problem Solving

Intelligent Path Planning for a Warehouse Delivery Robot

A. PEAS description

PEAS means Performance measure, Environment, Actuators, and Sensors. It is used to describe how an intelligent agent works.

Performance measure

The robot should:

- Deliver the package to cell (5,5).
- Start from cell (1,1).
- Avoid all blocked cells.
- Take the shortest possible route.
- Complete the delivery with minimum movement cost.
- Avoid collisions with shelves and other obstacles.

Environment

The environment is a warehouse represented by a 5 $\times$ 5 grid.

- Starting cell: (1,1)
- Destination cell: (5,5)
- Blocked cells: (2,2), (2,3), (3,3), (4,2), and (4,4)
- The robot can move only horizontally or vertically.
- Diagonal movement is not allowed.
- Each movement has a cost of 1.

The warehouse is a known, static, discrete, and fully observable environment because the positions of the shelves and the destination are already given.

Actuators

The actuators are the parts that make the robot move. They include:

- Left motor
- Right motor
- Wheels
- Braking or stopping mechanism

Using these actuators, the robot can move Up, Down, Left, or Right.

Sensors

The robot may use the following sensors:

- Position sensor to know its current cell.
- Obstacle sensor to detect blocked cells.
- Camera or grid sensor to identify the path.
- Package sensor to confirm that the package is delivered.

Therefore, the robot uses its sensors to understand its position and the actuators to move towards the destination.

B. Type Of Agent

The most suitable agent for this problem is a goal-based agent.

The robot has a specific goal: it must reach cell (5,5) and deliver the package. To achieve this goal, it has to examine possible movements and select a safe path from (1,1) to (5,5).

A simple reflex agent is not the best choice because it only reacts to the current situation. For example, it may see an obstacle in front of it and move in another direction without remembering the complete route.

A model-based agent can remember the warehouse layout, but the main purpose of this problem is reaching a particular destination. A utility-based agent could consider additional factors such as battery level, time, and safety. However, these factors are not given in the question.

Hence, a goal-based agent is the best choice because it selects actions according to the final goal of delivering the package.

C. Search Problem Formulation

Initial state

$S_0 = (1,1)$

The robot starts at cell (1,1).

Goal state

$G = (5,5)$

The robot must reach cell (5,5).

Actions

The robot can perform:

$A = \{ \text{"Up"}, \text{"Down"}, \text{"Left"}, \text{"Right"} \}$

An action is allowed only if:

The new cell lies inside the 5×5 grid.

The new cell is not blocked.

Blocked cells

$B = \{ (2,2), (2,3), (3,3), (4,2), (4,4) \}$

Transition model

The transition model gives the new position after an action:

$\text{Result}((r,c),a) = (r',c')$

For example:

$\text{Result}((1,1), \text{"Right"}) = (1,2)$

$\text{Result}((1,1), \text{"Down"}) = (2,1)$

Moving into a blocked cell or outside the grid is not permitted.

Path cost

Each valid movement has a cost of 1:

$\text{Cost}(a) = 1$

Therefore:

Path " " Cost = "Number of movements"

D. Breadth-First Search

Assume the successor order is:

"Right", "Down", "Left", "Up" BFS uses a FIFO queue and explores all nodes at one depth before moving to the next depth.

The notation X^* indicates that X has already been expanded.

Step	Expanded node	Queue after expansion
0	—	[(1, 1)]
1	(1, 1)	[(1, 2), (2, 1)]
2	(1, 2)	[(2, 1), (1, 3)]
3	(2, 1)	[(1, 3), (3, 1)]
4	(1, 3)	[(3, 1), (1, 4)]
5	(3, 1)	[(1, 4), (3, 2), (4, 1)]
6	(1, 4)	[(3, 2), (4, 1), (1, 5), (2, 4)]
7	(3, 2)	[(4, 1), (1, 5), (2, 4)]
8	(4, 1)	[(1, 5), (2, 4), (5, 1)]
9	(1, 5)	[(2, 4), (5, 1), (2, 5)]
10	(2, 4)	[(5, 1), (2, 5), (3, 4)]
11	(5, 1)	[(2, 5), (3, 4), (5, 2)]
12	(2, 5)	[(3, 4), (5, 2), (3, 5)]
13	(3, 4)	[(5, 2), (3, 5)]
14	(5, 2)	[(3, 5), (5, 3)]
15	(3, 5)	[(5, 3), (4, 5)]
16	(5, 3)	[(4, 5)]
17	(4, 5)	[(5, 5)]
18	(5, 5)	Goal reached

BFS path

The shortest path found is:

(1,1)→(1,2)→(1,3)→(1,4)→(1,5)→(2,5)→(3,5)→(4,5)→(5,5)

Movement sequence

Right, Right, Right, Right, Down, Down, Down, Down

Total cost

There are 8 movements:

("BFS total cost" =8)

This is optimal because BFS always finds the shallowest solution when every action has the same cost.

Program

```
from collections import deque

rows = 5
columns = 5

start = (1, 1)
goal = (5, 5)

blocked = {
    (2, 2),
    (2, 3),
    (3, 3),
    (4, 2),
    (4, 4)
}

moves = [
    (0, 1),      # Right
    (1, 0),      # Down
    (0, -1),     # Left
    (-1, 0)      # Up
]

def is_valid(cell):
    row, column = cell
    return (
        1 <= row <= rows
        and 1 <= column <= columns
        and cell not in blocked
    )

def get_neighbours(cell):
    row, column = cell
    neighbours = []

    for row_change, column_change in moves:
        next_cell = (
            row + row_change,
            column + column_change
        )

        if is_valid(next_cell):
            neighbours.append(next_cell)

    return neighbours
```

```

def make_path(parent, cell):
    path = []

    while cell is not None:
        path.append(cell)
        cell = parent[cell]

    path.reverse()
    return path

def breadth_first_search():
    queue = deque([start])
    visited = {start}
    parent = {start: None}

    while queue:
        current = queue.popleft()

        if current == goal:
            return make_path(parent, current)

        for next_cell in get_neighbours(current):
            if next_cell not in visited:
                visited.add(next_cell)
                parent[next_cell] = current
                queue.append(next_cell)

    return None

path = breadth_first_search()

print("BFS Path:")
print(path)

print("BFS Cost:")
print(len(path) - 1)

```

Output

```

==== RESTART: C:/Users/arish/AppData/Local/Programs/Python/Python313/BFS.py ====
BFS Path:
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5)]
BFS Cost:
8
|

```

E. Depth-First Search

DFS uses a LIFO stack. Using the same successor order, the successors are pushed in reverse order so that Right is explored first.

The stack top is shown on the right.

Step	Expanded node	Stack after expansion
0	—	[(1, 1)]
1	(1, 1)	[(2, 1), (1, 2)]
2	(1, 2)	[(2, 1), (1, 3)]
3	(1, 3)	[(2, 1), (1, 4)]
4	(1, 4)	[(2, 1), (1, 5), (2, 4)]
5	(2, 4)	[(2, 1), (1, 5), (3, 4)]
6	(3, 4)	[(2, 1), (1, 5), (3, 5)]
7	(3, 5)	[(2, 1), (1, 5), (4, 5)]
8	(4, 5)	[(2, 1), (1, 5), (5, 5)]
9	(5, 5)	Goal reached

DFS path

The path found is:

(1,1)→(1,2)→(1,3)→(1,4)→(2,4)→(3,4)→(3,5)→(4,5)→(5,5)

Movement sequence

"Right, Right, Right, Down, Down, Right, Down, Down"

Total cost

There are 8 movements:

("DFS total cost" =8)

In this particular successor order, DFS also finds an 8-movement path. However, this does not mean DFS is always optimal. A different successor order could cause DFS to find a longer path first.

Program

```
rows = 5
columns = 5

start = (1, 1)
goal = (5, 5)

blocked = {
    (2, 2),
    (2, 3),
    (3, 3),
    (4, 2),
    (4, 4)
}

moves = [
    (0, 1),    # Right
    (1, 0),    # Down
    (0, -1),   # Left
    (-1, 0)    # Up
]

def is_valid(cell):
    row, column = cell

    return (
        1 <= row <= rows
        and 1 <= column <= columns
        and cell not in blocked
    )

def get_neighbours(cell):
    row, column = cell
    neighbours = []

    for row_change, column_change in moves:
        next_cell = (
            row + row_change,
            column + column_change
        )

        if is_valid(next_cell):
            neighbours.append(next_cell)

    return neighbours

def make_path(parent, cell):
    path = []
```

```

        neighbours.append(next_cell)

    return neighbours

def make_path(parent, cell):
    path = []

    while cell is not None:
        path.append(cell)
        cell = parent[cell]

    path.reverse()
    return path

def depth_first_search():
    stack = [start]
    visited = {start}
    parent = {start: None}

    while stack:
        current = stack.pop()

        if current == goal:
            return make_path(parent, current)

        neighbours = get_neighbours(current)

        # Reverse order makes Right get explored first
        for next_cell in reversed(neighbours):
            if next_cell not in visited:
                visited.add(next_cell)
                parent[next_cell] = current
                stack.append(next_cell)

    return None

path = depth_first_search()

print("DFS Path:")
print(path)

print("DFS Cost:")
print(len(path) - 1)

```

Output

```

==== RESTART: C:/Users/arish/AppData/Local/Programs/Python/Python313/DFS.py ====
DFS Path:
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5)]
DFS Cost:
8

```

F. Comparison Of Bfs And Dfs

Point	BFS	DFS
Completeness	BFS will find a solution if one exists in this finite grid.	DFS can find a solution in this finite grid if visited cells are recorded.
Optimality	BFS is optimal because every movement has the same cost.	DFS is not guaranteed to find the shortest path.
Time complexity	$O(b^d)$, where b is the branching factor and d is the depth of the shortest solution.	$O(b^m)$, where m is the maximum depth of the search tree.
Space complexity	$O(b^d)$. It may store many nodes in the queue.	$O(bm)$. It generally stores fewer nodes.
Search method	Searches level by level.	Searches one branch deeply before backtracking.
Result in this problem	Finds an 8-movement shortest path.	Finds an 8-movement path for the selected movement order.
Main advantage	Guarantees the shortest route.	Uses less memory.
Main disadvantage	Requires more memory.	May return a longer route.

Conclusion

The robot successfully finds a path from (1, 1) to (5, 5) while avoiding blocked cells. BFS is the better choice because it guarantees the shortest path with minimum cost. Although DFS also finds a path, it may not always be the shortest.